

#Task 1: Dataset Understanding

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Text preprocessing
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Machine learning
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Deep learning
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import LabelEncoder

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
True

```
import pandas as pd
from google.colab import files
uploaded = files.upload()
df = pd.read_csv('customer_support_text_classification.csv')
print(df.head())
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving customer_support_text_classification.csv to customer_support_text_classification (1).csv

	ticket_id	channel	customer_message	\
0	TKT00001	chat	I need information about the payment process.	...
1	TKT00002	phone	I need information about the payment process.	
2	TKT00003	email	The refund process was fast and convenient.	I ...
3	TKT00004	social	My refund is still pending and this experience...	
4	TKT00005	chat	Please tell me how to update my account details.	

	sentiment_label	word_count	urgent_flag
0	neutral	18	1
1	neutral	7	0
2	positive	12	0
3	negative	15	1

```
print("Number of Records:", len(df))
```

Number of Records: 1500

```
print("Target Classes:")
print(df['sentiment_label'].unique())
```

Target Classes:
['neutral' 'positive' 'negative']

```
print(df['customer_message'].head())
```

0 I need information about the payment process. ...
1 I need information about the payment process.
2 The refund process was fast and convenient. I ...
3 My refund is still pending and this experience...
4 Please tell me how to update my account details.
Name: customer_message, dtype: object

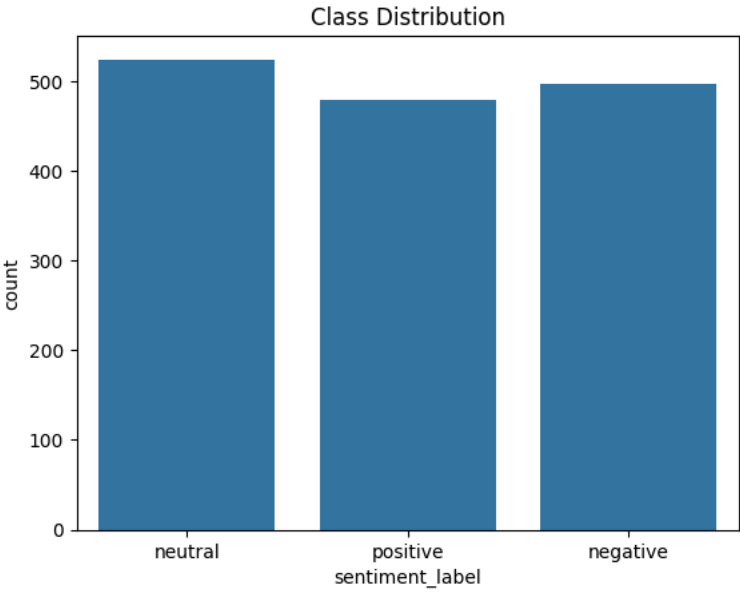
```
# Average number of words
avg_length = df['customer_message'].apply(lambda x: len(str(x).split())).mean()
print("Average Text Length:", avg_length)
```

Average Text Length: 12.722666666666667

```
print(df['sentiment_label'].value_counts())

# Plot class distribution
sns.countplot(x='sentiment_label', data=df)
plt.title('Class Distribution')
plt.show()
```

```
sentiment_label
neutral      524
negative     497
positive     479
Name: count, dtype: int64
```



#Task 2: Text Preprocessing

```
stop_words = set(stopwords.words('english'))

# Function for cleaning text
def clean_text(text):
    # Lowercase
    text = text.lower()

    # Remove special characters
    text = re.sub(r'[^a-zA-Z ]', '', text)

    # Tokenization
    words = word_tokenize(text)

    # Remove stopwords
    words = [word for word in words if word not in stop_words]

    return ' '.join(words)
```

```
# Create cleaned text column
df['cleaned_text'] = df['customer_message'].apply(clean_text)

# Display sample cleaned text
print(df[['customer_message', 'cleaned_text']].head())
```

```
customer_message \
0  I need information about the payment process. ...
1    I need information about the payment process.
2  The refund process was fast and convenient. I ...
3  My refund is still pending and this experience...
4   Please tell me how to update my account details.

cleaned_text
0  need information payment process ticket number...
1    need information payment process
2  refund process fast convenient appreciate quic...
3  refund still pending experience frustrating ti...
4    please tell update account details
```

#Task 3: Text Vectorization

```
# Input and output
X = df['cleaned_text']
y = df['sentiment_label']

# TF-IDF vectorizer
vectorizer = TfidfVectorizer(max_features=5000)

# Convert text into numerical vectors
X_vectorized = vectorizer.fit_transform(X)

print(X_vectorized.shape)
```

(1500, 146)

#Why Text Must Be Converted into Vectors:
#Text data cannot be directly understood by machine learning models because models work only with numbers. Vectorization conver
#TF-IDF gives importance to words that appear frequently in one document but not in all documents.

#Task 4: Baseline Model

```
X_train, X_test, y_train, y_test = train_test_split(
    X_vectorized,
    y,
    test_size=0.2,
```

```
        random_state=42
    )
```

```
# Create model
model = LogisticRegression()

# Train model
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

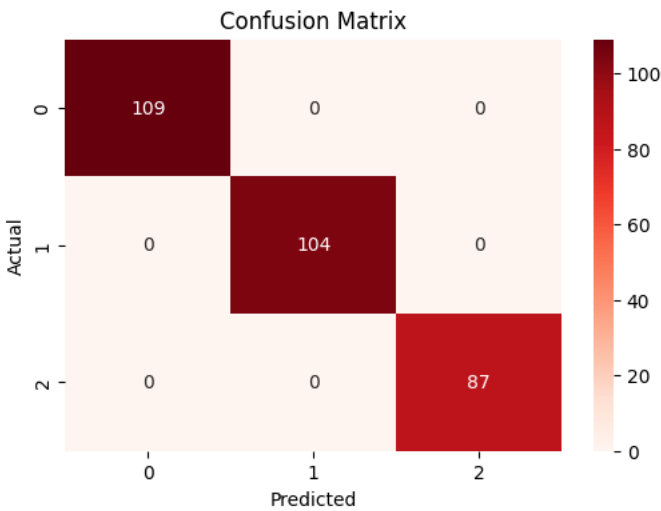
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Classification report
print(classification_report(y_test, y_pred))
```

Accuracy: 1.0				
	precision	recall	f1-score	support
negative	1.00	1.00	1.00	109
neutral	1.00	1.00	1.00	104
positive	1.00	1.00	1.00	87
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



#Task 5: Sequence Model Using LSTM

```
# Encode target labels
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

# One-hot encoding
y_categorical = to_categorical(y_encoded)
```

```
# Tokenizer
tokenizer = Tokenizer(num_words=5000)
tokenizer.fit_on_texts(df['cleaned_text'])

# Convert text into sequences
sequences = tokenizer.texts_to_sequences(df['cleaned_text'])

# Padding sequences
max_length = 20
X_padded = pad_sequences(sequences, maxlen=max_length, padding='post')

print(X_padded.shape)
```

(1500, 20)

```
X_train, X_test, y_train, y_test = train_test_split(
    X_padded,
    y_categorical,
    test_size=0.2,
    random_state=42
)
```

```
model = Sequential()

# Embedding layer
model.add(Embedding(input_dim=5000, output_dim=64, input_length=max_length))
```

```
# LSTM layer
model.add(LSTM(64))

# Output layer
model.add(Dense(3, activation='softmax'))

# Compile model
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# Model summary
model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input\_length` is deprecated
warnings.warn(
Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)

```
history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_data=(X_test, y_test)
)
```

Epoch 1/5
38/38 ————— 14s 91ms/step - accuracy: 0.4583 - loss: 1.0012 - val_accuracy: 0.6433 - val_loss: 0.5279
Epoch 2/5
38/38 ————— 4s 69ms/step - accuracy: 0.8050 - loss: 0.3682 - val_accuracy: 1.0000 - val_loss: 0.0469
Epoch 3/5
38/38 ————— 4s 50ms/step - accuracy: 1.0000 - loss: 0.0105 - val_accuracy: 1.0000 - val_loss: 0.0034
Epoch 4/5
38/38 ————— 2s 47ms/step - accuracy: 1.0000 - loss: 0.0026 - val_accuracy: 1.0000 - val_loss: 0.0021
Epoch 5/5
38/38 ————— 1s 31ms/step - accuracy: 1.0000 - loss: 0.0017 - val_accuracy: 1.0000 - val_loss: 0.0015

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)
```

10/10 ————— 0s 17ms/step - accuracy: 1.0000 - loss: 0.0015
Test Accuracy: 1.0

```
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

